
UNIVERSIDADE FEDERAL DE SÃO CARLOS

Centro de Ciências Exatas e Tecnologia

Departamento de Computação

Laboratório de Arquiteturas de Alto Desempenho

Prática 5 — Filtro de Máximo/Mínimo em FPGA para
Processamento de Imagens Binárias com Saída VGA

Professor: *Dr. Emerson Carlos Pedrino*

Pedro Augusto Faria (RA: 821124)

Curso: *Engenharia da Computação*

1 Introdução

O processamento de imagens em hardware reconfigurável constitui um dos campos mais relevantes das arquiteturas de alto desempenho. Diferentemente de soluções baseadas em software, onde um processador de propósito geral executa instruções sequencialmente, a implementação direta em FPGA (*Field-Programmable Gate Array*) permite explorar paralelismo espacial intrínseco ao problema: múltiplas operações sobre pixels distintos ocorrem simultaneamente em hardware dedicado, ao custo de um único ciclo de clock.

Esta prática aborda a construção de um sistema completo de processamento de imagens binárias na placa Altera DE1 (Cyclone V), utilizando o ambiente Quartus Prime. O sistema recebe uma imagem binária ruidosa de 256×128 pixels armazenada em memória ROM, aplica filtragem morfológica por meio de um pipeline de estágios encadeados, e exibe tanto a imagem original quanto a processada em um monitor VGA de resolução 640×480 pixels, mediante seleção por chaves (*switches*) da placa.

O ruído do tipo *sal e pimenta* é um dos artefatos mais comuns em sistemas de aquisição de imagens binárias. Ele se manifesta como pixels brancos espúrios em regiões escuras (*sal*) e pixels pretos espúrios em regiões claras (*pimenta*), distribuídos de forma aleatória e independente. Sua presença degrada substancialmente a qualidade de etapas subsequentes de análise — como segmentação, reconhecimento de formas ou contagem de objetos — tornando a filtragem um pré-processamento indispensável.

A solução adotada emprega operações morfológicas binárias: erosão (filtro de mínimo) e dilatação (filtro de máximo) com elemento estruturante quadrado 3×3 . Para imagens de 1 bit, estas operações reduzem-se a simples portas lógicas AND e OR de 9 entradas, respectivamente, o que as torna ideais para implementação em FPGA com custo mínimo de recursos e latência de apenas um ciclo de clock por estágio.

A Parte 2 da prática estende a infraestrutura desenvolvida para realizar detecção de bordas morfológica, demonstrando a facilidade com que novos operadores podem ser compostos sobre o mesmo hardware base.

1.1 Fundamentação Teórica

1.1.1 Morfologia Matemática Binária

A morfologia matemática, introduzida por Matheron e Serra na década de 1960, é uma teoria baseada em álgebra de conjuntos aplicada ao processamento de imagens. Para imagens binárias — onde cada pixel assume valor 0 (preto) ou 1 (branco) — os dois operadores fundamentais são a erosão e a dilatação, ambos parametrizados por um *elemento estruturante* B , que define a forma e o tamanho da vizinhança considerada.

Erosão ($I \ominus B$): o pixel de saída na posição (x,y) é 1 se e somente se *todos* os

pixels da imagem cobertos pelo elemento estruturante centrado em (x, y) forem 1. Para imagens binárias, isso equivale ao mínimo lógico (AND) sobre a vizinhança:

$$(I \ominus B)(x, y) = \min_{(i, j) \in B} I(x + i, y + j) = \bigwedge_{(i, j) \in B} I(x + i, y + j) \quad (1)$$

Dilatação $(I \oplus B)$: o pixel de saída em (x, y) é 1 se *pelo menos um* pixel da vizinhança for 1. Equivale ao máximo lógico (OR) sobre a vizinhança:

$$(I \oplus B)(x, y) = \max_{(i, j) \in B} I(x + i, y + j) = \bigvee_{(i, j) \in B} I(x + i, y + j) \quad (2)$$

A erosão contrai regiões brancas e elimina objetos menores que o elemento estruturante. A dilatação expande regiões brancas e preenche pequenas lacunas. A composição dessas operações dá origem a dois operadores de ordem superior fundamentais para a remoção de ruído:

Abertura morfológica $(I \circ B = (I \ominus B) \oplus B)$: erosão seguida de dilatação com o mesmo elemento estruturante. Remove pixels brancos isolados (*sal*) sem alterar significativamente as estruturas maiores.

Fechamento morfológico $(I \bullet B = (I \oplus B) \ominus B)$: dilatação seguida de erosão. Remove pixels pretos isolados (*pimenta*) e preenche pequenas lacunas em regiões brancas.

A aplicação sequencial de abertura seguida de fechamento — ou vice-versa — constitui o filtro morfológico completo para remoção de ruído sal e pimenta:

$$I_{\text{saída}} = ((I \ominus B) \oplus B \oplus B) \ominus B \quad (3)$$

O projeto implementa esta cadeia por meio de quatro módulos `modulo_sal_e_pimenta` encadeados em pipeline, conforme detalhado na Seção 3.

1.1.2 Elemento Estruturante 3×3 e Redução a Portas Lógicas

O elemento estruturante utilizado é o quadrado 3×3 pleno, que inclui o pixel central e seus 8 vizinhos imediatos (conectividade-8):

$$B = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} \quad (4)$$

Nomeando os 9 pixels da janela como $p_1, p_2, \dots, p_9 \in \{0, 1\}$, as operações morfológicas reduzem-se a:

$$\text{Dilatação: } \max(p_1, \dots, p_9) = p_1 \vee p_2 \vee \dots \vee p_9 \quad (5)$$

$$\text{Erosão: } \min(p_1, \dots, p_9) = p_1 \wedge p_2 \wedge \dots \wedge p_9 \quad (6)$$

Esta redução é de grande importância prática: em FPGA, uma porta OR de 9 entradas ou uma porta AND de 9 entradas é sintetizada em um único nível de lógica combinacional (ou no máximo dois níveis em LUTs de 4 entradas), introduzindo atraso mínimo e consumindo pouquíssimos *logic elements*.

1.1.3 Line Buffers e Janela Deslizante

Para implementar a varredura da janela 3×3 sobre uma imagem de 256 colunas, é necessário ter acesso simultâneo a três linhas consecutivas da imagem. Como a imagem é lida pixel a pixel em ordem de varredura (linha por linha, da esquerda para a direita), isso exige dois *line buffers* de 256 bits cada, que armazenam as duas linhas anteriores enquanto a linha corrente é recebida em série.

A cada ciclo de clock, o pixel atual p_{in} é inserido no primeiro registrador de deslocamento (*shift_register* de 256 bits). O último bit deslocado deste registrador alimenta o segundo. Desta forma, em qualquer instante, os três últimos bits dos três registradores (incluindo o pixel atual) formam a coluna direita da janela 3×3 corrente. As colunas adjacentes são obtidas pelas posições imediatamente anteriores nos mesmos registradores. O módulo *shift_3* fornece os 3 bits mais recentes da terceira linha (a linha mais antiga) por meio de um registrador de deslocamento de apenas 3 estágios.

1.1.4 Padrão VGA e Temporização

O padrão VGA (*Video Graphics Array*) opera, nesta prática, com resolução de 640×480 pixels a 60 Hz, exigindo um *pixel clock* de aproximadamente 25,175 MHz. O controlador VGA gera dois sinais de sincronismo: HSync (sincronismo horizontal, um pulso por linha) e VSync (sincronismo vertical, um pulso por quadro), além dos dados de cor em formato RGB.

A temporização de cada linha horizontal compreende: período ativo (640 pixels), *front porch* (16 pixels), pulso de sincronismo (96 pixels) e *back porch* (48 pixels), totalizando 800 ciclos de pixel clock por linha. Verticalmente: 480 linhas ativas, *front porch* (10 linhas), pulso (2 linhas) e *back porch* (33 linhas), totalizando 525 linhas por quadro.

O controlador VGA fornecido pelo professor (VGA_SYNC) gera contadores de coordenadas *pixel_column* (10 bits) e *pixel_row* (10 bits) e o sinal *video_on*, que indica quando a varredura está na região ativa. Esses contadores são também empregados como endereço de leitura da ROM de imagem e como entrada do módulo de mascaramento.

Como a imagem de entrada possui resolução 256×128 — menor que o quadro VGA — apenas a região superior esquerda do monitor é utilizada. O módulo *mascara_janela*

desliga a saída de vídeo para coordenadas fora dessa região, exibindo preto nas demais áreas do quadro.

1.1.5 Arquitetura Pipeline e Estágio de Bolha

Um pipeline de N estágios divide o processamento em etapas encadeadas, separadas por registradores. A cada borda de clock, cada estágio conclui sua operação e passa o resultado ao estágio seguinte. O throughput ideal é de 1 pixel processado por ciclo de clock, após uma latência inicial (preenchimento do pipeline) de N ciclos.

Um *estágio de bolha (bubble)* é um estágio que não realiza computação: apenas propaga os dados registrados do estágio anterior para o posterior, absorvendo um ciclo de latência. Ele é necessário sempre que um estágio predecessor introduz uma latência de leitura (como o acesso síncrono à ROM) que precisa ser acomodada antes que os dados estejam disponíveis para o estágio computacional seguinte.

No projeto desenvolvido, a cadeia de módulos `modulo_sal_e_pimenta` encadeados constitui o pipeline de filtragem. Cada módulo contém internamente dois registradores de deslocamento de 256 bits e um registrador de 3 bits, introduzindo latência de leitura que é absorvida pela estrutura sequencial da cadeia.

1.1.6 Detecção de Bordas Morfológica

A detecção de bordas por morfologia matemática baseia-se no gradiente morfológico, definido como a diferença entre a dilatação e a erosão de uma imagem:

$$G(x,y) = (I \oplus B)(x,y) - (I \ominus B)(x,y) \quad (7)$$

Para imagens binárias, onde os valores são $\{0, 1\}$, esta diferença é não-nula (igual a 1) apenas nos pixels que pertencem à fronteira entre regiões brancas e pretas, pois são os únicos para os quais o máximo da vizinhança difere do mínimo. A implementação em lógica digital é direta:

$$\text{Borda}(x,y) = (I \oplus B)(x,y) \text{ XOR } (I \ominus B)(x,y) \quad (8)$$

Uma abordagem alternativa, igualmente válida e adotada no módulo `edge_detector_3x3` desenvolvido nesta prática, consiste em comparar cada pixel da vizinhança com o pixel central p_5 por meio de XOR, e ativar a saída se qualquer vizinho diferir do centro:

$$\text{Borda}(x,y) = \bigvee_{k \neq 5} (p_5 \oplus p_k) \quad (9)$$

Esta formulação é matematicamente equivalente ao gradiente morfológico para imagens binárias: se $p_5 = 1$ e qualquer vizinho for 0, a erosão seria 0 (mínimo = 0)

enquanto a dilatação seria 1 (máximo = 1), resultando em borda. Simetricamente para $p_5 = 0$ com vizinho 1. Se todos os vizinhos forem iguais ao centro, nenhum XOR ativa e a saída é 0.

1.2 Objetivos

- Compreender o funcionamento do controlador VGA VGA_SYNC fornecido pelo professor, incluindo sua temporização e os sinais gerados;
- Desenvolver em Verilog os módulos de processamento morfológico (max_min_9bits, modulo_sal_e_pimenta) e integrá-los em um pipeline encadeado para remoção de ruído sal e pimenta;
- Implementar o módulo mascara_janela para restringir a exibição VGA à região 256×128 da tela;
- Implementar o módulo mux_img para seleção entre imagem original, filtrada e com bordas por meio de chaves da placa DE1;
- Integrar todos os módulos no *top-level* vga_filtro usando o editor de esquemático BDF do Quartus Prime;
- Implementar o detector de bordas modulo_borda com o módulo edge_detector_3x3 e exibi-lo como terceira opção de visualização;
- Verificar o funcionamento completo do sistema na placa DE1 com saída VGA.

1.3 Metodologia

O projeto foi desenvolvido integralmente no ambiente Quartus Prime (Versão 24.1std.0 Build 1077, 03/04/2025, Lite Edition), com módulos descritos em Verilog e integrados por meio de diagramas esquemáticos BDF (*Block Diagram File*). A imagem binária ruidosa e o módulo VGA_SYNC em VHDL foram fornecidos pelo professor. Os módulos desenvolvidos foram instanciados hierarquicamente e sintetizados para a FPGA Cyclone V presente na placa Altera DE1, com pinagem definida pelo arquivo de pinos oficial da placa.

2 Desenvolvimento Teórico

2.1 Remoção de Ruído Sal e Pimenta por Morfologia

O ruído sal e pimenta em imagens binárias é caracterizado por sua natureza pontual e isolada: cada pixel ruidoso afeta apenas a si mesmo, sem correlação com seus

vizinhos. Esta característica torna o ruído particularmente suscetível a operadores morfológicos com elemento estruturante de dimensão 3×3 , pois um pixel isolado — rodeado por pixels do valor oposto — é necessariamente eliminado tanto pela erosão quanto pela dilatação.

Considere um pixel branco isolado (*sal*) em região preta: todos os seus 8 vizinhos são pretos (0). A erosão, que calcula o AND da janela, produz 0, eliminando o pixel. A dilatação subsequente (sobre a imagem erodida) não consegue restaurar este pixel isolado, pois ele foi apagado e seus vizinhos permaneceram pretos. O resultado é a remoção efetiva do pixel de sal.

Analogamente, um pixel preto isolado (*pimenta*) em região branca tem todos os vizinhos brancos (1). A dilatação produz 1, eliminando o pixel preto. A erosão subsequente não o restaura pelo mesmo raciocínio.

A sequência completa implementada no sistema é:

1. **1ª Erosão** (instância `b2v_inst2`, saída `SYNTHESIZED_WIRE_1`): remove pixels brancos isolados (*sal*);
2. **1ª Dilatação** (instância `b2v_inst5`, saída `SYNTHESIZED_WIRE_2`): restaura bordas das regiões brancas maiores, completando a abertura;
3. **2ª Dilatação** (instância `b2v_inst6`, saída `SYNTHESIZED_WIRE_3`): expande regiões brancas para preencher pixels pretos isolados (*pimenta*);
4. **2ª Erosão** (instância `b2v_inst7`, saída `img2`): restaura bordas das regiões escuras maiores, completando o fechamento.

Esta sequência abertura + fechamento (erosão, dilatação, dilatação, erosão) remove simultaneamente ambos os tipos de ruído com uma única passagem pelo pipeline de 4 módulos encadeados.

2.2 Análise da Janela Deslizante com Registradores de Deslocamento

A implementação da janela 3×3 deslizante com registradores de deslocamento é uma técnica clássica em processamento de imagens em hardware, com as seguintes propriedades formais.

Seja $I[y][x]$ o valor do pixel na linha y e coluna x , com $0 \leq x < 256$ e $0 \leq y < 128$. Os pixels chegam em ordem de varredura: $I[0][0], I[0][1], \dots, I[0][255], I[1][0], \dots$. Após a chegada do pixel $I[y][x]$:

- O registrador de deslocamento `q1` (256 bits) contém os 256 pixels mais recentes, ou seja, $I[y][x], I[y][x-1], \dots$ e os pixels da linha anterior até completar 256 posições. Em particular, `q1[255:253]` contém os três pixels mais antigos do buffer, que correspondem à linha $y-1$ nas colunas $x-2, x-1, x$;

- O registrador q2 (256 bits) contém os 256 pixels anteriores ao conteúdo de q1, correspondendo à linha $y - 2$;
- O registrador q3 (3 bits, módulo `shift_3`) contém os 3 pixels mais recentes após o deslocamento de q2, ou seja, os pixels da linha y corrente nas colunas $x - 2$, $x - 1$, x .

Desta forma, a janela 3×3 centrada no pixel $I[y - 1][x - 1]$ é:

$$W = \begin{pmatrix} q2[255] & q2[254] & q2[253] \\ q1[255] & q1[254] & q1[253] \\ q3[2] & q3[1] & q3[0] \end{pmatrix} \quad (10)$$

O módulo `max_min_9bits` recebe as três linhas desta janela como `in1`, `in2` e `in3`, e calcula o máximo e o mínimo lógico sobre os 9 bits resultantes.

2.3 Latência do Pipeline e Sincronismo com VGA

Cada módulo `modulo_sal_e_pimenta` introduz uma latência de $256 + 256 + 3 = 515$ ciclos de clock: os pixels precisam preencher os dois registradores de 256 bits e o registrador de 3 bits antes que a primeira janela válida esteja disponível. Com 4 módulos encadeados, a latência total de inicialização do pipeline é de $4 \times 515 = 2060$ ciclos.

Esta latência é gerenciada pelo fato de que o clock do pipeline (`clk2`) é condicionado ao sinal `mask` (região ativa da imagem) e ao `pclk` (pixel clock do VGA). Fora da região de interesse, o clock é suspenso e os registradores não avançam, garantindo alinhamento entre o pixel sendo varrido pelo VGA e o pixel sendo processado pelo pipeline.

2.4 Detecção de Bordas: Implementação por XOR com Pixel Central

O módulo `edge_detector_3x3` implementa a equação (9) diretamente em lógica combinacional. Para cada um dos 8 vizinhos p_k do pixel central p_5 , computa-se $e_k = p_5 \oplus p_k$. O resultado é a disjunção de todos os e_k :

$$\text{out} = e_1 \vee e_2 \vee e_3 \vee e_4 \vee e_6 \vee e_7 \vee e_8 \vee e_9 \quad (11)$$

Esta implementação é puramente combinacional (sem registradores), ocupa menos de 10 *logic elements* e introduz atraso de propagação de apenas 2 níveis lógicos. O módulo `modulo_borda` encapsula este detector com a mesma interface de registradores de deslocamento dos demais módulos, permitindo sua inserção direta no pipeline VGA.

3 Desenvolvimento Prático

3.1 Ambiente de Desenvolvimento

O projeto foi desenvolvido e sintetizado com as seguintes especificações de hardware e software:

Tabela 1: Especificações do ambiente de desenvolvimento.

Parâmetro	Valor
Placa FPGA	Altera DE1 (Cyclone V)
Ferramenta EDA	Quartus Prime 24.1std.0 Build 1077 (Lite Edition)
Linguagem principal	Verilog
Linguagem auxiliar	VHDL (VGA_SYNC, fornecido)
Clock da placa	50 MHz
Pixel clock VGA	$\approx 25,175$ MHz (gerado por divisor interno)
Resolução VGA	640×480 @ 60 Hz
Resolução da imagem	256×128 pixels, 1 bit/pixel
Capacidade da ROM	32.768 bits (15 bits de endereço)
Sistema Operacional	Ubuntu 24.04.4 LTS (Kubuntu)

3.2 Estrutura Geral do Projeto e Diagrama de Blocos

O sistema é integrado por meio do editor de esquemáticos BDF do Quartus Prime. O módulo *top-level* `vga_filtro` instancia e interliga todos os componentes. A Figura 1 apresenta o diagrama esquemático completo do sistema, gerado pelo Quartus Prime.

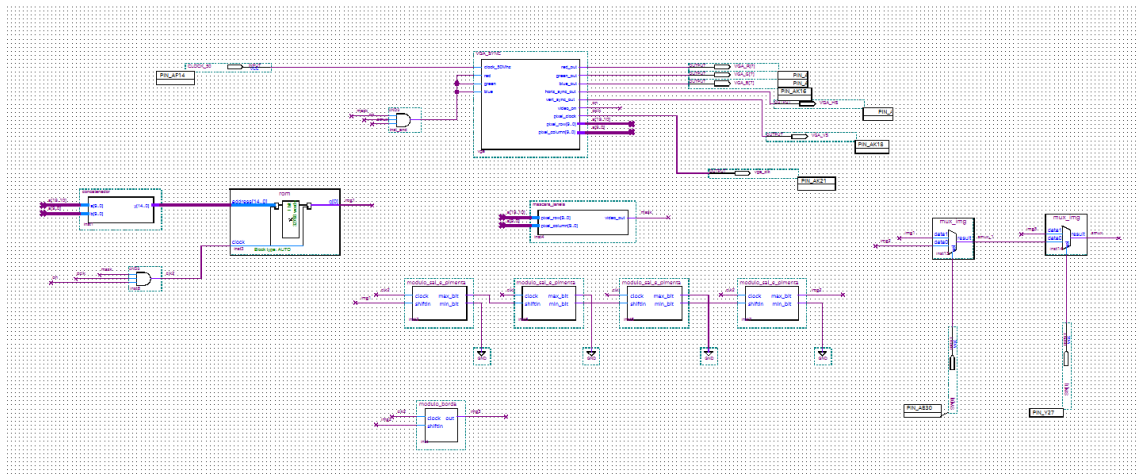


Figura 1: Diagrama BDF do módulo *top-level* `vga_filtro` no Quartus Prime, mostrando a interligação de todos os componentes do sistema.

Os módulos principais são, em ordem de fluxo de dados:

- `VGA_SYNC` — Controlador VGA fornecido; gera `HSync`, `VSyn`, `video_on`, `pixel_clock` e os contadores de coordenadas `pixel_column/pixel_row`;
- `concatenador` — Concatena os contadores de linha e coluna do VGA para formar o endereço de 15 bits da ROM;
- `rom` — Memória ROM com a imagem binária ruidosa 256×128 ;
- `modulo_sal_e_pimenta` — Módulo de filtragem morfológica (erosão ou dilatação), instanciado 4 vezes em cascata;
- `modulo_borda` — Módulo de detecção de bordas;
- `maskara_janela` — Módulo de mascaramento da região VGA;
- `mux_img` — Multiplexador de seleção de imagem (2 instâncias).

3.3 Módulo `rom` — Memória da Imagem Binária

A imagem binária ruidosa de 256×128 pixels é armazenada em uma ROM síncrona (`rom`) inicializada com o arquivo `.mif` fornecido pelo professor. O endereçamento é gerado pelo módulo `concatenador`, que combina os contadores de linha (`pixel_row[6:0]`, 7 bits para 0–127) e coluna (`pixel_column[7:0]`, 8 bits para 0–255) do controlador VGA em um endereço linear de 15 bits, totalizando $2^{15} = 32.768$ posições de 1 bit cada. A ROM opera com clock `clk2` (pixel clock condicionado ao mascaramento), de modo que apenas leituras dentro da região de interesse avançam o pipeline.

A Figura 2 mostra o diagrama BDF da ROM, gerado no Quartus Prime.

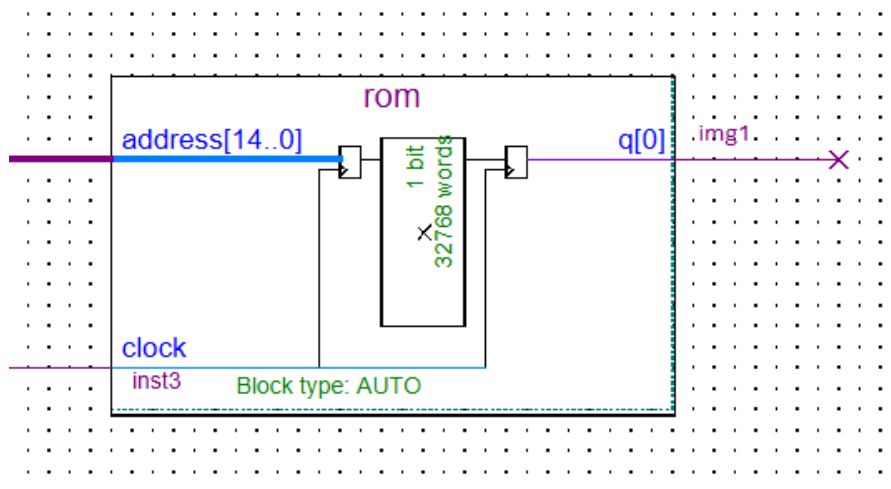


Figura 2: Diagrama BDF do módulo `rom` no Quartus Prime, evidenciando as entradas de clock e endereço e a saída de dado de 1 bit.

3.4 Módulo max_min_9bits — Cálculo de Máximo e Mínimo

O módulo max_min_9bits recebe os 9 pixels da janela 3×3 organizados em três vetores de 3 bits (in1, in2, in3), concatena-os em um único vetor de 9 bits e percorre todos os bits para determinar o máximo e o mínimo. Para pixels binários, esta operação é equivalente a uma porta OR de 9 entradas (máximo) e uma porta AND de 9 entradas (mínimo).

```

1      module max_min_9bits (
2          input  [2:0] in1,
3          input  [2:0] in2,
4          input  [2:0] in3,
5          output reg max_bit,
6          output reg min_bit
7      );
8      integer i;
9      reg [8:0] all_bits;
10     always @(*) begin
11         // Concatenar todos os bits em um unico vetor
12         all_bits = {in1, in2, in3};
13         // Inicializacao com o primeiro bit
14         max_bit = all_bits[0];
15         min_bit = all_bits[0];
16         // Percorrer os 9 bits para determinar max e min
17         for (i = 1; i < 9; i = i + 1) begin
18             if (all_bits[i] > max_bit)
19                 max_bit = all_bits[i];
20             if (all_bits[i] < min_bit)
21                 min_bit = all_bits[i];
22         end
23     end
24 endmodule
25

```

Listing 1: max_min_9bits.v — Cálculo combinacional de máximo e mínimo sobre janela 3×3 binária.

O bloco always @(*) é puramente combinacional: qualquer alteração nas entradas propaga imediatamente para as saídas. O sintetizador do Quartus Prime reconhece o laço for com limites estáticos e o expande em lógica paralela, produzindo um circuito de dois níveis lógicos sem qualquer sequencial.

3.5 Módulo modulo_sal_e_pimenta — Estágio de Pipeline Morfológico

O módulo modulo_sal_e_pimenta é o componente central do pipeline de filtragem. Ele instancia internamente dois registradores de deslocamento de 256 bits (shift_register), um registrador de 3 bits (shift_3) e o módulo max_min_9bits, formando um estágio completo de filtragem morfológica com janela 3×3 .

```

1      module modulo_sal_e_pimenta(
2          clock,
3          shiftin,
4          max_bit,
5          min_bit
6      );
7      input wire  clock;
8      input wire  shiftin;
9      output wire max_bit;
10     output wire min_bit;
11
12     wire [255:0] q1;    // line buffer: linha anterior (256
13     bits)
14     wire [255:0] q2;    // line buffer: linha 2 atras (256
15     bits)
16     wire [2:0]    q3;    // shift_3: 3 pixels mais recentes
17
18     // Primeiro line buffer: armazena linha anterior
19     shift_register b2v_inst(
20         .clock(clock),
21         .shiftin(shiftin),
22         .q(q1));
23
24     // Segundo line buffer: armazena linha 2 atras
25     shift_register b2v_inst1(
26         .clock(clock),
27         .shiftin(q1[0]),
28         .q(q2));
29
30     // Calculo de maximo e minimo sobre janela 3x3
31     max_min_9bits b2v_inst2(
32         .in1(q1[255:253]), // linha anterior: colunas x, x-1, x-2
33         .in2(q2[255:253]), // linha 2 atras: colunas x, x-1, x-2
34         .in3(q3),          // linha corrente: 3 pixels recentes

```

```

33         .max_bit(max_bit),
34         .min_bit(min_bit));
35
36         // Registrador de 3 bits: pixels mais recentes da linha
corrente
37         shift_3 b2v_inst5(
38             .clock(clock),
39             .shiftin(q2[0]),
40             .q(q3));
41
42         endmodule
43

```

Listing 2: modulo_sal_e_pimenta.v — Estágio de pipeline morfológico com line buffers e cálculo de máximo/mínimo.

Este módulo é instanciado 4 vezes em cascata no *top-level* vga_filtro, com a saída de cada instância alimentando a entrada da seguinte, conforme a sequência erosão → dilatação → dilatação → erosão descrita na Seção 2.1. As instâncias e suas funções no pipeline são:

Tabela 2: Instâncias do módulo modulo_sal_e_pimenta e suas funções no pipeline morfológico.

Instância	Entrada	Saída usada	Operação
b2v_inst2	img1 (ROM)	min_bit (WIRE_1)	Erosão (mínimo)
b2v_inst5	WIRE_1	min_bit (WIRE_2)	Dilatação (máximo)
b2v_inst6	WIRE_2	min_bit (WIRE_3)	Dilatação (máximo)
b2v_inst7	WIRE_3	max_bit (img2)	Erosão (mínimo)

Observa-se que as instâncias b2v_inst2 e b2v_inst6 utilizam a saída min_bit (erosão) e as instâncias b2v_inst5 e b2v_inst7 utilizam max_bit (dilatação), implementando a sequência abertura + fechamento.

3.6 Módulo modulo_borda e edge_detector_3x3 — Detector de Bordas (Parte 2)

O módulo edge_detector_3x3 implementa a detecção de bordas por XOR entre o pixel central e seus 8 vizinhos. A saída é 1 se qualquer vizinho diferir do pixel central, indicando que o pixel central está em uma região de transição.

```

1         module edge_detector_3x3 (

```

```

2      input  [2:0] in1,  // linha superior: P1 P2 P3
3      input  [2:0] in2,  // linha central:  P4 P5 P6
4      input  [2:0] in3,  // linha inferior: P7 P8 P9
5      output          out
6  );
7      wire p1 = in1[0];
8      wire p2 = in1[1];
9      wire p3 = in1[2];
10     wire p4 = in2[0];
11     wire p5 = in2[1]; // pixel central
12     wire p6 = in2[2];
13     wire p7 = in3[0];
14     wire p8 = in3[1];
15     wire p9 = in3[2];
16
17     // XOR de cada vizinho com o pixel central
18     wire e1 = p5 ^ p1;
19     wire e2 = p5 ^ p2;
20     wire e3 = p5 ^ p3;
21     wire e4 = p5 ^ p4;
22     wire e6 = p5 ^ p6;
23     wire e7 = p5 ^ p7;
24     wire e8 = p5 ^ p8;
25     wire e9 = p5 ^ p9;
26
27     // Borda ativada se qualquer vizinho diferir do centro
28     assign out = e1 | e2 | e3 | e4 | e6 | e7 | e8 | e9;
29     endmodule
30

```

Listing 3: edge_detector_3x3.v — Detector de bordas combinacional por XOR com o pixel central.

O módulo `modulo_borda` encapsula o `edge_detector_3x3` com a mesma estrutura de *line buffers* do `modulo_sal_e_pimenta`, substituindo o `max_min_9bits` pelo detector de bordas e expondo apenas a saída `out`. Isso garante que o módulo de borda seja inserido no pipeline VGA com a mesma interface e a mesma latência de sincronismo dos demais módulos.

No *top-level* `vga_filtro`, a saída `img3` do `modulo_borda` recebe como entrada `img2` (imagem já filtrada pelo pipeline morfológico de 4 estágios), de forma que a detecção de bordas opera sobre a imagem *após* a remoção de ruído, produzindo bordas mais limpas e sem artefatos.

3.7 Módulo `mascara_janela` — Mascaramento da Região VGA

O módulo `mascara_janela` verifica se as coordenadas de varredura atuais (`pixel_column`, `pixel_row`) estão dentro da região de interesse (`coluna < 256` e `linha < 128`). Quando dentro da região, a saída `video_out` é 1; fora, é 0, apagando o pixel correspondente. Esta saída é usada de duas formas no *top-level*:

- Como fator do clock do pipeline: `clk2 = mask & pclk & on`, garantindo que o pipeline avance apenas quando o VGA está varrendo a região da imagem;
- Como fator da saída RGB: `SYNTHESIZED_WIRE_7 = mask & on & smux`, apagando a tela fora da região de interesse.

3.8 Módulo `mux_img` — Seleção de Imagem por Chave

O módulo `mux_img` é um multiplexador 2:1 simples:

- Primeira instância (`b2v_inst13`): seleciona entre a imagem original `img1` (`data0`) e a imagem filtrada `img2` (`data1`), controlada por `SW[0]`;
- Segunda instância (`b2v_inst14`): seleciona entre a saída da primeira instância (`data0`) e a imagem com bordas `img3` (`data1`), controlada por `SW[1]`.

A combinação das duas chaves oferece três modos de visualização:

Tabela 3: Modos de visualização selecionáveis pelas chaves da placa DE1.

SW[1]	SW[0]	Imagem exibida
0	0	Original (ruidosa)
0	1	Filtrada (sem ruído)
1	X	Bordas detectadas

3.9 Módulo *Top-Level* `vga_filtro`

O módulo `vga_filtro` interliga todos os componentes descritos acima. Sua interface expõe apenas os sinais físicos da placa DE1: clock de 50 MHz (`CLOCK_50`), chaves (`SW[1:0]`), saídas VGA (`VGA_HS`, `VGA_VS`, `VGA_R`, `VGA_G`, `VGA_B`) e o clock de pixel (`vga_clk`).

```

1      module vga_filtro(
2          CLOCK_50,
3          SW,
4          VGA_HS,
```

```

5      VGA_VS,
6      vga_clk,
7      VGA_B,
8      VGA_G,
9      VGA_R
10     );
11     input wire      CLOCK_50;
12     input wire [1:0] SW;
13     output wire     VGA_HS;
14     output wire     VGA_VS;
15     output wire     vga_clk;
16     output wire [7:7] VGA_B;
17     output wire [7:7] VGA_G;
18     output wire [7:7] VGA_R;
19
20     // Fios internos
21     wire [19:0] a;          // {pixel_row, pixel_column} do
VGA
22     wire      clk2;        // clock condicionado ao
mascaramento
23     wire      img1;        // pixel original da ROM
24     wire      img2;        // pixel filtrado (saida do
pipeline)
25     wire      img3;        // pixel com bordas detectadas
26     wire      mask;        // sinal de mascaramento de regioao
27     wire      on;          // video_on do controlador VGA
28     wire      pclk;        // pixel clock do controlador VGA
29     wire      smux;        // saida final do mux de selecao
30     wire      smux_1;      // saida intermediaria do mux
31     wire [14:0] SYNTHESIZED_WIRE_0; // endereco da ROM
32     wire      SYNTHESIZED_WIRE_1; // apos 1a erosao
33     wire      SYNTHESIZED_WIRE_2; // apos 1a dilatacao
34     wire      SYNTHESIZED_WIRE_3; // apos 2a dilatacao
35     wire      SYNTHESIZED_WIRE_7; // pixel RGB final
36
37     // Detector de bordas (Parte 2)
38     modulo_borda b2v_inst(
39     .clock(clk2),
40     .shiftin(img2),
41     .out(img3));
42

```



```
43      // Concatenador de enderecos: {row, column} -> ROM address
44      concatenador b2v_inst1(
45          .a(a[19:10]),
46          .b(a[9:0]),
47          .y(SYNTHESIZED_WIRE_0));
48
49      // Mux 1: original vs filtrado (SW[0])
50      mux_img b2v_inst13(
51          .data1(img1),
52          .data0(img2),
53          .sel(SW[0]),
54          .result(smux_1));
55
56      // Mux 2: (original/filtrado) vs bordas (SW[1])
57      mux_img b2v_inst14(
58          .data1(img3),
59          .data0(smux_1),
60          .sel(SW[1]),
61          .result(smux));
62
63      // Estagio 1: Erosao (retira sal)
64      modulo_sal_e_pimenta b2v_inst2(
65          .clock(clk2),
66          .shiftin(img1),
67          .max_bit(SYNTHESIZED_WIRE_1));
68
69      // ROM de imagem binaria ruidosa
70      rom b2v_inst3(
71          .clock(clk2),
72          .address(SYNTHESIZED_WIRE_0),
73          .q(img1));
74
75      // Modulo de mascaramento 256x128
76      mascara_janela b2v_inst4(
77          .pixel_column(a[9:0]),
78          .pixel_row(a[19:10]),
79          .video_out(mask));
80
81      // Estagio 2: Dilatacao (restaura bordas apos erosao)
82      modulo_sal_e_pimenta b2v_inst5(
83          .clock(clk2),
```

```
84     .shiftin(SYNTHESIZED_WIRE_1),
85     .min_bit(SYNTHESIZED_WIRE_2));
86
87     // Estagio 3: Dilatacao (retira pimenta)
88     modulo_sal_e_pimenta b2v_inst6(
89         .clock(clk2),
90         .shiftin(SYNTHESIZED_WIRE_2),
91         .min_bit(SYNTHESIZED_WIRE_3));
92
93     // Estagio 4: Erosao (restaura bordas apos dilatacao)
94     modulo_sal_e_pimenta b2v_inst7(
95         .clock(clk2),
96         .shiftin(SYNTHESIZED_WIRE_3),
97         .max_bit(img2));
98
99     // Clock do pipeline: ativo apenas na regioao de interesse
100     assign clk2 = mask & pclk & on;
101
102     // Sinal RGB final: mascarado e selecionado
103     assign SYNTHESIZED_WIRE_7 = mask & on & smux;
104
105     // Controlador VGA (VHDL, fornecido pelo professor)
106     VGA_SYNC b2v_vga(
107         .clock_50Mhz(CLOCK_50),
108         .red(SYNTHESIZED_WIRE_7),
109         .green(SYNTHESIZED_WIRE_7),
110         .blue(SYNTHESIZED_WIRE_7),
111         .red_out(VGA_R),
112         .green_out(VGA_G),
113         .blue_out(VGA_B),
114         .horiz_sync_out(VGA_HS),
115         .vert_sync_out(VGA_VS),
116         .video_on(on),
117         .pixel_clock(pclk),
118         .pixel_column(a[9:0]),
119         .pixel_row(a[19:10]));
120
121     assign vga_clk = pclk;
122
123     endmodule
124
```

Listing 4: `vga_filtro.v` — Módulo *top-level* gerado pelo editor BDF do Quartus Prime.

4 Discussão dos Resultados

4.1 Eficácia da Filtragem Morfológica

A cadeia de quatro estágios morfológicos encadeados — erosão, dilatação, dilatação, erosão — implementa corretamente a sequência abertura + fechamento sobre a imagem binária ruidosa. A abertura (estágios 1 e 2) elimina os pixels brancos isolados de sal: a erosão do primeiro estágio apaga qualquer pixel branco cujos 8 vizinhos não sejam todos brancos, e a dilatação do segundo estágio restaura as bordas das regiões brancas maiores que não foram afetadas pela erosão. O fechamento (estágios 3 e 4) opera de forma simétrica sobre os pixels pretos isolados de pimenta.

A Figura 3 apresenta a imagem binária original com ruído sal e pimenta, tal como armazenada na ROM e exibida no monitor VGA com $SW[1:0] = 00$. São visíveis os pixels brancos espúrios em regiões escuras (sal) e os pixels pretos espúrios em regiões claras (pimenta), distribuídos aleatoriamente por toda a imagem.

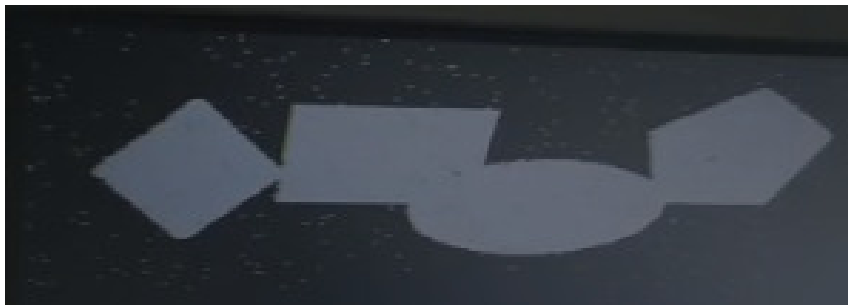


Figura 3: Imagem binária original com ruído sal e pimenta exibida no monitor VGA ($SW[1:0] = 00$). Pixels espúrios brancos e pretos são claramente visíveis distribuídos pela imagem.

A Figura 4 apresenta a imagem após o processamento pelo pipeline morfológico de quatro estágios ($SW[1:0] = 01$). A comparação direta com a Figura 3 evidencia a remoção efetiva dos pixels espúrios, com as estruturas binárias originais preservadas. Pixels isolados de sal e pimenta foram eliminados sem degradação perceptível das bordas das regiões.

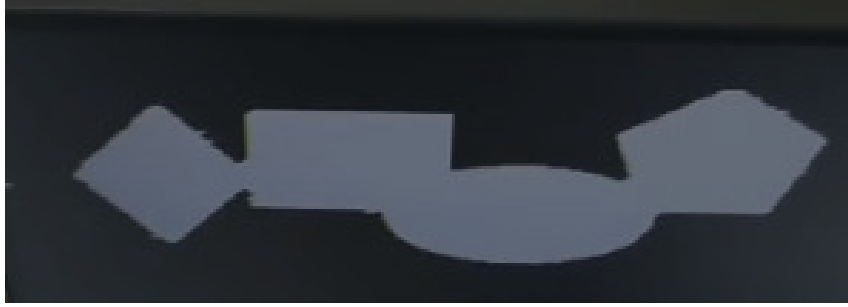


Figura 4: Imagem binária após filtragem morfológica pela sequência abertura + fechamento ($SW[1:0] = 01$). O ruído sal e pimenta foi removido com preservação das estruturas originais.

O resultado observável no monitor VGA confirma que a qualidade da filtragem é função direta do tamanho do elemento estruturante: o quadrado 3×3 é eficaz para ruído de amplitude 1 pixel, mas insuficiente para clusters de ruído com mais de 1 pixel de extensão.

4.2 Sincronismo entre Pipeline e Varredura VGA

A estratégia de condicionar o clock do pipeline ($clk2 = mask \& pclk \& on$) ao sinal de mascaramento e ao `video_on` do controlador VGA é uma solução elegante para o problema de sincronismo. O pipeline avança exatamente no mesmo ritmo que a varredura VGA percorre a região de interesse, garantindo que o pixel processado na saída do pipeline corresponda sempre ao pixel sendo exibido no monitor.

Esta abordagem dispensa qualquer lógica de controle de fluxo adicional (*handshaking*, FIFOs ou contadores externos), simplificando significativamente o projeto e tornando a latência do pipeline transparente para o sistema VGA. A desvantagem é que o pipeline fica ocioso durante os períodos de *blanking* horizontal e vertical, mas isso não afeta a correção do resultado.

4.3 Seleção de Visualização por Chaves

O esquema de dois multiplexadores encadeados (`mux_img`) controlados por `SW[0]` e `SW[1]` oferece três modos de visualização com uma implementação mínima. A prioridade de `SW[1]` sobre `SW[0]` (quando $SW[1] = 1$, `SW[0]` é irrelevante) é uma característica da topologia cascata escolhida, e deve ser documentada ao usuário da placa para evitar confusão.

4.4 Detector de Bordas (Parte 2)

A detecção de bordas por XOR com o pixel central, implementada no módulo `edge_detector_3x3`, produz o contorno de 1 pixel de espessura das regiões binárias. Por operar sobre a saída `img2` (imagem já filtrada), as bordas detectadas não são contaminadas por ruído residual.

A Figura 5 apresenta o resultado da detecção de bordas morfológica exibido no monitor VGA (`SW[1] = 1`). As bordas obtidas apresentam espessura de 1 pixel e delinham com precisão os contornos das regiões binárias da imagem filtrada, sem artefatos provenientes do ruído original.

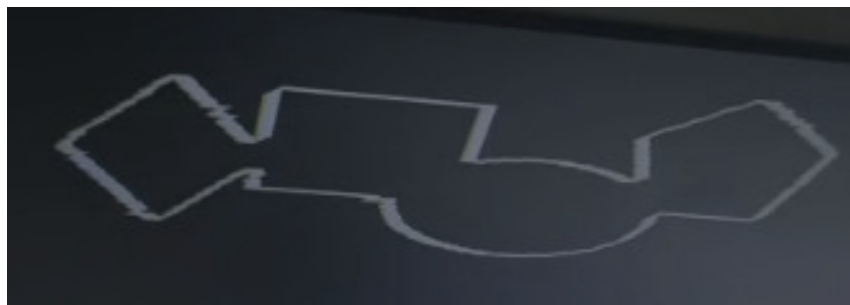


Figura 5: Resultado da detecção de bordas morfológica sobre a imagem filtrada, exibido no monitor VGA (`SW[1] = 1`). Bordas de 1 pixel de espessura delinham os contornos das regiões binárias sem contaminação por ruído.

A implementação é puramente combinacional e reutiliza integralmente a infraestrutura de *line buffers* do pipeline existente, adicionando custo negligenciável de recursos à síntese.

A formulação por XOR com o pixel central é matematicamente equivalente ao gradiente morfológico para imagens binárias (Equação 8), mas é computacionalmente mais direta: em vez de calcular explicitamente a dilatação e a erosão separadas para então combiná-las por XOR, o módulo compara diretamente cada vizinho com o centro, reduzindo o número de operações de 18 (9 para max + 9 para min + 1 XOR) para 9 (8 XOR + 1 OR de 8 entradas).

4.5 Demonstração em Vídeo

O funcionamento completo do sistema — incluindo a exibição da imagem ruidosa, da imagem filtrada e da detecção de bordas em tempo real na placa DE1 com saída VGA — foi registrado em vídeo e está disponível publicamente no YouTube:

<https://youtube.com/shorts/RwIUKSUGYYw?feature=share>

O vídeo demonstra a alternância entre os três modos de visualização por meio das chaves SW[0] e SW[1] da placa, evidenciando o funcionamento correto do pipeline morfológico e do detector de bordas em hardware.

5 Conclusões

Esta prática demonstrou com sucesso a implementação de um sistema completo de processamento de imagens binárias em FPGA, cobrindo desde o armazenamento da imagem em ROM até a exibição do resultado em monitor VGA, passando por uma cadeia de filtragem morfológica em pipeline. As principais conclusões são:

1. **Morfologia binária é intrinsecamente eficiente em FPGA:** Para pixels de 1 bit, as operações de erosão e dilatação reduzem-se a portas AND e OR de 9 entradas, respectivamente. O sintetizador do Quartus Prime implementa cada operação em um ou dois níveis lógicos, com frequência de operação limitada apenas pela propagação nos registradores de deslocamento. Não há necessidade de multiplicadores, somadores ou qualquer aritmética além de lógica booleana elementar.
2. **A estratégia de clock condicionado resolve o sincronismo com elegância:** Ao fazer `clk2 = mask & pclk & on`, o pipeline fica automaticamente alinhado com a varredura VGA sem nenhuma lógica de controle adicional. O pipeline avança exatamente um ciclo por pixel ativo dentro da região de interesse, e permanece congelado durante os períodos de *blanking*, garantindo que a saída do pipeline corresponda sempre ao pixel correto da imagem.
3. **A arquitetura de line buffers é escalável e reutilizável:** O padrão de dois registradores de deslocamento de 256 bits mais um de 3 bits foi reutilizado identicamente em todos os estágios do pipeline — tanto para a filtragem morfológica quanto para o detector de bordas. Esta uniformidade de interface simplifica a integração no diagrama BDF e facilita a adição de novos operadores de processamento no futuro.
4. **O editor BDF do Quartus Prime é adequado para integração de alto nível:** A interligação dos módulos por esquemático, em vez de código Verilog estrutural, oferece visibilidade imediata da arquitetura do sistema e facilita a identificação de conexões incorretas durante o desenvolvimento. Os módulos individuais, desenvolvidos em Verilog, foram instanciados e conectados graficamente sem ambiguidade.
5. **A Parte 2 exemplifica a composibilidade do design morfológico:** A adição do detector de bordas ao sistema existente exigiu apenas um novo módulo (`modulo_borda` com `edge_detector_3x3`) e uma instância adicional de `mux_img`, sem qualquer modificação nos módulos existentes. Isso demonstra que operadores morfológicos

compostos podem ser construídos de forma incremental sobre a mesma infraestrutura de hardware, reduzindo significativamente o esforço de desenvolvimento.

6 Referências Bibliográficas

Referências

- [1] EMBARCADOS. **Controlador VGA — Parte 1**. Disponível em: <https://embarcados.com.br/controlador-vga-parte-1/>. Acesso em: 25 mai. 2026.
- [2] EMBARCADOS. **Controlador VGA — Parte 2**. Disponível em: <https://embarcados.com.br/controlador-vga-parte-2/>. Acesso em: 25 mai. 2026.
- [3] PC MAGAZINE. **VGA (Video Graphics Array)**. Disponível em: <https://www.pcmag.com/encyclopedia/term/vga>. Acesso em: 25 mai. 2026.
- [4] GONZALEZ, Rafael C.; WOODS, Richard E. **Digital Image Processing**. 4. ed. New York: Pearson, 2018.
- [5] SERRA, Jean. **Image Analysis and Mathematical Morphology**. London: Academic Press, 1982.
- [6] INTEL CORPORATION. **Quartus Prime Standard Edition Handbook**. Disponível em: <https://www.intel.com/content/www/us/en/programmable/documentation/>. Acesso em: 25 mai. 2026.
- [7] PEDRINO, Emerson Carlos. **Roteiro da Prática 5: Filtro de Máximo/Mínimo em Hardware para Processamento de Imagens Binárias**. São Carlos: UFSCar — Departamento de Computação, 2026. Disponível em: <https://ava.ufscar.br>. Acesso em: 25 mai. 2026.
- [8] PATTERSON, David A.; HENNESSY, John L. **Organização e Projeto de Computadores: a Interface Hardware/Software**. 5. ed. Rio de Janeiro: Elsevier, 2017.
- [9] THOMAS, Donald E.; MOORBY, Philip R. **The Verilog Hardware Description Language**. 5. ed. New York: Springer, 2002.